Trabajo Práctico N.º 8 — Food Store en la feria

> **Base de Datos II · UTN FRM · Tecnicatura Universitaria en Programación** — Comisión 4 · Docente: Sergio Neira · Ciclo 2026 **Integrantes:** Mateo Lautaro Liendo, Lucas Ávila y Amanda Pagano · **Fecha:** 04/11/2026
> **Repositorio:** github.com/ALucrecia-Pagano/Bases-de-datos-II-NoSQL (carpeta `TP8_Unidad8/`, etiqueta `tp8`)
> Informe de las consignas 1.a, 2.c, 3.a, 3.b y 3.c. Los scripts, la spec de Kiro, la bitácora de IA y la DUIA están en el repositorio. **Regla de oro:** el móvil propone, el servidor dispone; precio, stock y ventas los decide PostgreSQL.

1.a · Qué datos viajan a la tablet

La tablet recibe una copia mínima del catálogo para consultar productos y armar borradores sin conexión. La copia no reemplaza al servidor: PostgreSQL sigue siendo la fuente de verdad para el precio vigente, el stock, la disponibilidad y cualquier venta.

Dato de `producto`	Viaja a SQLite	Representación en `producto_cache`	Motivo
`id`	Sí	`id`	Identificar el producto en los ítems del borrador.
`nombre`	Sí	`nombre`	Mostrar el producto en la consulta local.
`precio_lista`	Sí, como precio observado	`precio_centavos`	Armar el borrador sin usar `REAL`; se conserva como entero en centavos.
`stock`	Sí, como observación	`stock_visto`	Permitir una validación preliminar offline.
`activo`	Sí, como disponibilidad	`disponible`	Ocultar productos que el servidor marcó como no disponibles.
Baja lógica	Sí, si corresponde	`eliminado`	Identificar productos retirados en una instantánea posterior.
Fecha de copia	Sí	`fecha_copia`	Registrar cuándo se obtuvo la instantánea del servidor.
`id_categoria`	No	—	No es necesario para consultar ni armar el borrador.
`descripcion`	No	—	No es necesaria para las reglas de sincronización de este práctico.
`created_at`	No	—	Es un metadato que no interviene en la operación offline.

`precio_centavos` y `stock_visto` representan valores observados, no autorizaciones. Al sincronizar, el servidor vuelve a comparar cada solicitud con una instantánea de PostgreSQL antes de clasificarla.

2.c · Casos C2 a C6

Los casos se prueban en una copia SQLite basada en la instantánea de PostgreSQL. Cada caso usa datos ficticios y solicitudes con UUID diferentes, excepto C6, donde se intenta reenviar intencionalmente el mismo UUID.

Caso	Preparación	Resultado esperado
C2	En la copia de PostgreSQL se sube el precio del producto 1 (1050,00 → 1100,00) y se exporta una instantánea nueva. Luego el cliente acepta: se actualiza el precio del borrador y se envía una solicitud nueva.	Primera solicitud `CONFLICTO`; nueva solicitud `SIMULADO_APROBADO`.
C3	Se solicita una cantidad mayor que el stock del producto 2.	`RECHAZADO` por stock insuficiente.
C4	En la copia de PostgreSQL se da de baja el producto 3 (`activo = false`) y se exporta una instantánea nueva.	`RECHAZADO` con motivo `Producto no disponible`. Al recargar el catálogo, el producto ya no aparece, pero su fila sigue en `producto_cache`.
C5	Se crea una solicitud con precio, disponibilidad y cantidad compatibles.	`SIMULADO_APROBADO`.
C6	Se intenta insertar nuevamente una solicitud con el mismo UUID.	Error de clave primaria; no se duplica ni se reevalúa la solicitud anterior.

Cómo se preparan y ejecutan los casos

1. `postgresql/verificar_sin_cambios_2c.sql` en la copia `foodstore_tp8_feria`: registra la cantidad de pedidos y el stock antes de empezar.
2. `postgresql/preparar_casos_2c.sql`: en la copia del servidor sube el precio del producto 1 (C2), da de baja el producto 3 (C4) y exporta `movil/servidor_snapshot_2c.csv`. Tiene un control que impide ejecutarlo en otra base.
3. `casos_2c.sql` en SQLite: crea los borradores con la instantánea de 2.a, importa la instantánea nueva con `movil/recargar_snapshot.sql` y corre el semáforo.
4. `postgresql/verificar_sin_cambios_2c.sql` otra vez: la cantidad de pedidos y el stock deben ser iguales al paso 1.

La tablet nunca escribe en PostgreSQL. Los únicos cambios en la copia del servidor son el precio (C2) y la baja lógica (C4), hechos por el script de preparación, que representa al administrador del servidor.

Por qué C4 da “Producto no disponible”

La vista `v_snapshot_movil` deriva `disponible` y `eliminado` de la misma columna (`activo`). Una baja lógica en PostgreSQL exporta `disponible = f` y `eliminado = t`, y el semáforo, que revisa la disponibilidad antes que la baja, registra ese motivo. El estado es el que pide la consigna: `RECHAZADO`.

Dónde iría la protección real contra el reenvío (C6)

En la tablet, la clave primaria de `cola_sync` solo evita duplicados en la cola local. En un sistema real, la protección tiene que estar en el servidor: PostgreSQL guardaría el `solicitud_uuid` de cada solicitud recibida con una restricción `UNIQUE`, en la misma transacción que crea el pedido. Así, un reenvío del mismo UUID no genera un segundo pedido ni descuenta stock dos veces.

Ejecución y resultados

Ejecutada por Amanda Pagano con SQLite CLI 3.54.1 (base en memoria) y PostgreSQL en la copia `foodstore_tp8_feria`, creada a partir de `foodstore_dev`. Esta sección complementa la ejecución anterior: ahora los cambios de C2 y C4 se hicieron en la copia del servidor y se exportó `servidor_snapshot_2c.csv`.

Comando (desde la raíz del repositorio):

```
        sqlite3 :memory: ".read TP8_Unidad8/movil/schema_sqlite.sql" ".import --csv --skip 1 TP8_Unidad8/movil/servidor_snaps
hot.csv snapshot_servidor_import" ".read TP8_Unidad8/movil/importar_snapshot.sql" ".read TP8_Unidad8/movil/casos_2c.sql"
```

Caso	Solicitud	Estado	Motivo
C2	`c2-solicitud-1`	`CONFLICTO`	Cambió el precio observado
C2 (aceptado)	`c2-solicitud-2`	`SIMULADO_APROBADO`	Precio, disponibilidad y stock coinciden
C3	`c3-solicitud`	`RECHAZADO`	Stock insuficiente
C4	`c4-solicitud`	`RECHAZADO`	Producto no disponible
C5	`c5-solicitud`	`SIMULADO_APROBADO`	Precio, disponibilidad y stock coinciden
C6	reenvío de `c5-solicitud`	Error `UNIQUE constraint failed: cola_sync.solicitud_uuid`	La tercera ejecución del semáforo deja los cinco estados sin cambios

C4, catálogo recargado. El catálogo disponible muestra solo la Muzzarella (110000 centavos, stock 50) y la Napolitana (150000 centavos, stock 35). La fila de la Coca 1.5L sigue en `producto_cache` con `disponible = 0` y `eliminado = 1`: es una baja lógica, no un borrado.

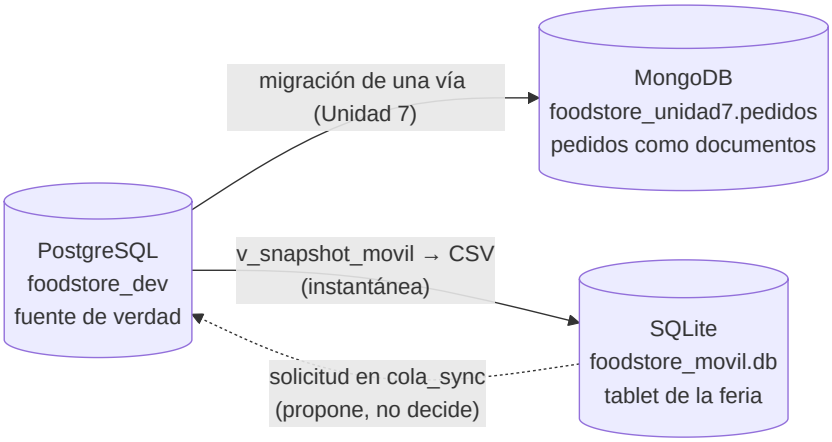
PostgreSQL no cambió (`postgresql/verificar_sin_cambios_2c.sql`):

Medición	Antes (20:47)	Después (21:01)
Cantidad de pedidos	5	5
Cantidad de detalles de pedido	7	7
Stock de los productos 1, 2 y 3	50 / 35 / 100	50 / 35 / 100
Stock total	185	185

El precio del producto 1 y el estado `activo` del producto 3 sí cambiaron en la copia: los cambió el script de preparación (el servidor), no la tablet.

3.a · Mapa de la capa de datos

Food Store usa tres motores. Cada uno tiene un papel distinto, pero solo uno decide: **PostgreSQL es la fuente de verdad**. MongoDB y SQLite guardan copias derivadas de él, con propósitos diferentes.



Mapa de la capa de datos de Food Store

Qué guarda cada motor

Motor	Qué guarda	Para qué se usa	¿Manda?
PostgreSQL (`foodstore_dev`)	Catálogo (`producto`, `categoria`), clientes y usuarios, pedidos y `detalle_pedido`, stock y precios vigentes.	Operación del negocio: registrar ventas, mantener precios y stock con restricciones, claves foráneas y transacciones.	**Sí.** Es la única fuente de verdad.
MongoDB (`foodstore_unidad7.pedidos`)	Cinco pedidos históricos migrados, cada uno con sus líneas embebidas, precio histórico de venta (Decimal128) y cliente reducido a ID y alias sintético.	Consultar un pedido completo en un solo documento, sin joins.	**No.** Es una copia de lectura; si difiere de PostgreSQL, vale PostgreSQL.
SQLite (tablet)	`producto_cache` (instantánea del catálogo), `pedido_borrador` e `item_borrador`, `cola_sync` y `snapshot_servidor`.	Consultar el catálogo y armar borradores en la feria, sin señal.	**Solo sobre sus borradores**, mientras no se envían. Precio y stock son valores observados.

Quién manda sobre cada dato

Dato	Decide	Copia en MongoDB	Copia en SQLite
Precio vigente	PostgreSQL (`producto.precio_lista`)	No se migró	`precio_centavos`, como precio observado
Stock	PostgreSQL (`producto.stock`)	No se migró	`stock_visto`, solo para validar de forma preliminar
Disponibilidad y baja lógica	PostgreSQL (`producto.activo`)	No se migró	`disponible` y `eliminado`, derivados de `activo`
Venta confirmada (pedido)	PostgreSQL (`pedido`, `detalle_pedido`)	Sí, como documento histórico	**Nunca.** La tablet solo arma borradores
Precio histórico de venta	PostgreSQL (`detalle_pedido.precio_unitario`)	Sí (`lineas[].precio_unitario`)	No
Borrador del pedido	SQLite, hasta que se envía	No	`pedido_borrador`, `item_borrador`
Resultado de una solicitud	PostgreSQL, al sincronizar	No	`cola_sync` guarda la simulación; `SIMULADO_APROBADO` no es una venta
Datos de clientes	PostgreSQL	Solo ID y alias sintético	No viajan datos personales

La regla que une los tres motores es la del práctico: **el móvil propone, el servidor dispone**. Los datos fluyen desde PostgreSQL hacia las copias; lo único que vuelve desde la tablet es una solicitud, y la decide PostgreSQL.

3.b · Plan de mantenimiento

Este agregado amplía el plan de mantenimiento, respaldo y recuperación de la Unidad 6 para incluir MongoDB y las tablets. El plan original no se modifica: sigue vigente para PostgreSQL (respaldo diario con `pg_dump -Fc`, restauración ensayada en `foodstore_restore_test`, RTO ≤ 15 min y RPO ≤ 24 h).

¿Qué se respalda y cómo se prueba la restauración?

Motor	Qué se respalda	Cómo se prueba
PostgreSQL	La base completa, todos los días, con `pg_dump -Fc` (plan de la Unidad 6). Mejora pendiente del mismo plan: roles y permisos con `pg_dumpall --globals-only`.	Restaurar en `foodstore_restore_test` y comparar conteos por tabla y los nueve indicadores de integridad (`restore_test.ps1`).
MongoDB	`foodstore_unidad7` con `mongodump`. No es crítico: es una copia derivada y se puede reconstruir desde PostgreSQL con el script de migración de la Unidad 7.	Restaurar con `mongorestore` en una base descartable y comprobar que haya 5 documentos y que cada `total` coincida con la suma de sus `lineas`.
SQLite (tablet)	Solo lo que no existe en otro lado: borradores y solicitudes sin enviar. Al cerrar cada jornada se copia la base con `.backup` de `sqlite3`, que genera una copia consistente aunque la base esté abierta. El catálogo no se respalda: se recarga con una instantánea nueva.	Abrir la copia, ejecutar `PRAGMA integrity_check;` (debe responder `ok`) y contar las solicitudes `PENDIENTE`.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

El servidor vuelve a un estado anterior, y la tablet puede tener una instantánea más nueva que él. Como el servidor manda, la tablet se adapta:

1. **Descarta su catálogo y pide una instantánea nueva** al servidor restaurado. No conserva precios ni stock propios: la `fecha_copia` de su copia puede ser posterior a la del respaldo restaurado.
2. **Vuelve a evaluar sus solicitudes `PENDIENTE`** con el semáforo contra esa instantánea. Si el precio volvió a un valor anterior, la solicitud pasa a `CONFLICTO` y se resuelve como en C2.
3. **Reenvía las solicitudes que se enviaron después del respaldo**, con su mismo `solicitud_uuid`. El servidor restaurado ya no las tiene, así que las evalúa de nuevo. Con la protección de C6 en el servidor, un reenvío nunca genera un pedido duplicado.

¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?

- **`CONFLICTO`:** el vendedor, en el momento de sincronizar, con el cliente presente. Si el cliente acepta el precio nuevo, se actualiza el borrador y se envía una solicitud nueva (C2).
- **`RECHAZADO`:** el encargado del puesto, al cierre de cada jornada de feria. Avisa al cliente o anula el borrador.
- **Una vez por semana**, el administrador de la base revisa los motivos que se repiten. Por ejemplo, muchos “Stock insuficiente” indican que la tablet sincroniza poco y hay que actualizar la instantánea más seguido.

Consulta de revisión en la tablet:

```sql
SELECT estado, motivo, COUNT(*) AS solicitudes
FROM cola_sync
WHERE estado IN ('CONFLICTO', 'RECHAZADO')
GROUP BY estado, motivo
ORDER BY estado, motivo;
```

Con los datos de la consigna 2.c devuelve una solicitud en `CONFLICTO` por cambio de precio y dos en `RECHAZADO`: una por producto no disponible y otra por stock insuficiente.

3.c · Code review con IA

Herramienta: Antigravity (modelo Claude Sonnet 5.5), en modo Plan, de solo lectura. La consigna menciona OpenCode; el docente autorizó usar cualquier herramienta de IA. **Alcance:** los 75 scripts (`.sql`, `.js`, `.ps1`, `.py`) de los dos repositorios de la cursada: `Base_de_datos2` (TP1 a TP7 y Unidad 6) y `Bases-de-datos-II-NoSQL` (Unidad 7 y Unidad 8). Se le dio una copia de los scripts y los `.md` en una carpeta aparte, con la prohibición de modificar archivos o conectarse a las bases. En una primera pasada leyó 54 scripts; se le pidió completar los 21 restantes hasta llegar a 75 de 75.

Criterio de decisión. *Aceptado*: el hallazgo es correcto. Si es de un TP ya calificado, no se modifica y queda registrado como mejora. *Corregido*: es correcto y se corrigió en este TP. *Descartado*: no aplica, con el fundamento indicado.

Tabla de hallazgos

ID	Ubicación	Hallazgo (resumido)	Decisión	Motivo
F01	TP2 · `restricciones_integridad.sql`	El trigger de stock lee sin `FOR UPDATE` y no descuenta stock: dos sesiones concurrentes pueden vender más que el stock.	Aceptado	Correcto por lectura del código. TP2 calificado: mejora registrada.

ID	Ubicación	Hallazgo (resumido)	Decisión	Motivo
F02	TP1 y TP5 · `schema.sql`	`DROP ... CASCADE` sin control de base.	Aceptado (parcial)	Falta el control de base. Pero la IA leyó mal `views.sql`: indica correrse *después* de `schema.sql` para armar una base desde cero, no correr `schema.sql` sobre la base de carga.
F03	TP7 · `fn_resetear_password.sql`	`rol_soporte` puede resetear la clave de un ADMIN.	Aceptado	Observación de diseño: el reseteo para soporte es un requisito del TP7, pero no excluye a los administradores.
F04	TP7 · `fn_autenticar.sql` / `fn_resetear_password.sql`	Una función recorta espacios de la clave (`btrim`) y la otra no.	Aceptado	Correcto por lectura del código; no se comprobó en la base.
F05	**TP2 · `restricciones_integridad.sql` y TP1 · `schema.sql`**	**Un pedido ENTREGADO se puede borrar y arrastra su detalle.**	**Aceptado**	**Comprobado en la base (ver desarrollo).**
F06	Unidad 7 · `prueba_validacion_expr_mongodb.js`	El validador solo controla la aritmética, no los rangos.	Aceptado	Correcto; ese script prueba una sola afirmación. Mejora: `$jsonSchema`.
F07	**Unidad 8 · `recargar_snapshot.sql`**	**Un error al recargar deja la tablet sin instantánea.**	**Corregido**	**Comprobado y corregido (ver desarrollo).**
F08	Unidad 8 · `importar_snapshot.sql`	`CAST` de texto no numérico da 0 sin error en SQLite.	Aceptado	Riesgo bajo: el CSV sale de una columna `bigint` de la vista. Se registra como mejora.
F09	**Unidad 8 · `recargar_snapshot.sql`**	**Un producto que desaparece del CSV quedaría disponible en la tablet.**	**Descartado**	**Comprobado en la base (ver desarrollo).**
F10	Unidad 6 · `backup_foodstore.ps1`	El conteo de control se mide en otra transacción que el respaldo.	Descartado	El informe de la Unidad 6 declara que no hubo escrituras durante el ensayo, condición necesaria para el problema. Válido para un servidor con uso real.
F11	TP7 · `activar_auditoria_local.sql`	`log_statement = 'all'` registra claves en texto plano.	Descartado	Decisión deliberada de laboratorio, documentada en el propio script, con claves sintéticas y restauración del valor al terminar.
F12	Unidad 6 · `restore_test.ps1`	La restauración descarta dueños y permisos.	Aceptado	Ya figura como mejora en el informe de la Unidad 6 (`pg_dumpall --globals-only`).
F13	TP7 · `fn_resetear_password.sql`	`gen_salt('bf')` usa el costo por defecto, bajo.	Aceptado	Correcto por lectura del código; mejora registrada.
F14	TP7 · `fn_autenticar.sql`	Diferencia de tiempo entre mail existente e inexistente.	Aceptado	Riesgo bajo y difícil de medir con costo bajo; mejora registrada.
F15	TP4 · `consulta_competencia.sql`	Usa `now()`: el resultado depende del día de ejecución.	Aceptado	Problema de reproducibilidad; TP6 ya usa una fecha fija.
F16	TP3 · `seed_masivo.sql`	Carga masiva sin control de base ni `ON_ERROR_STOP`.	Aceptado	Correcto; los scripts posteriores (TP6, TP7) ya incorporan ese control.
F17	TP3 · `seed_masivo.sql` y TP2	Cantidades y stock se generan sin relación entre sí.	Aceptado	Riesgo latente: solo afecta si se instalan los triggers de TP2, que hoy no están en ninguna base.
F18	TP3 · `verificacion_carga.sql`	El rango esperado en un comentario no coincide con la carga real.	Aceptado	Comentario desactualizado; el valor verificado sí es correcto.
F19	TP3 · `indices_propuestos.sql`	Crea los 4 índices aunque 3 se revirtieron.	Aceptado	Registro histórico; si se ejecutara tal cual, chocaría con TP5.
F20	TP3 · `optimizacion_propuesta_2.sql`	La Propuesta 2 no se mide combinada con la 1, como dice el comentario.	Aceptado	El comentario no coincide con el script.
F21	TP5 · `medir_q2_rondas.sql`	No se puede repetir con el índice ya aplicado.	Aceptado	Script de medición ya ejecutado y archivado.
F22	TP5 · `medir_q4_rondas.sql`	`DROP/CREATE INDEX` con `COMMIT` sin control de base.	Aceptado	Correcto; el propio archivo advierte el estado del índice.

ID	Ubicación	Hallazgo (resumido)	Decisión	Motivo
F23	TP3 y TP5 · consultas	`BETWEEN` con `timestamptz` excluye el último día después de las 00:00.	Aceptado	Buena práctica; con los datos de la carga no cambia el resultado.
F24	TP7 · `usuario_anon_cobertura.sql`	`LOCK TABLE` sin `NOWAIT` ni `lock_timeout`.	Aceptado	Riesgo bajo en un laboratorio de un solo usuario.

Desarrollo de tres hallazgos

F05 — Un pedido ENTREGADO se puede borrar (TP2) · Aceptado

Qué dijo la IA. El trigger de TP2 que protege los pedidos en estado final solo actúa en `UPDATE`, y la clave foránea `pedido → detalle_pedido` es `ON DELETE CASCADE`. Un `DELETE` borra el pedido y su detalle, contra la regla de preservar la historia de ventas.

Cómo se comprobó. En la copia `foodstore_tp8_feria`, dentro de `BEGIN … ROLLBACK`: se instalaron los triggers de TP2 y se verificó que existían (2 en `pedido`). Un `UPDATE` del pedido 1 (ENTREGADO) a CANCELADO fue rechazado por el trigger. El `DELETE` del mismo pedido respondió `DELETE 1`, y su detalle pasó de 2 filas a 0. Después del `ROLLBACK` quedaron 5 pedidos y 0 triggers. La prueba se repitió en `foodstore_copia_trabajo`, la base donde TP2 instaló los triggers: hoy no los tiene, y el `DELETE` también pasa.

Decisión. Aceptado. TP2 está calificado y no se modifica. Mejora registrada: un trigger `BEFORE DELETE` que rechace los pedidos en estado final, y `ON DELETE RESTRICT` en la clave foránea.

F07 — Un error al recargar vacía la instantánea de la tablet (Unidad 8) · Corregido

Qué dijo la IA. En `recargar_snapshot.sql`, si el `INSERT` en `snapshot_servidor` falla, `sqlite3` sigue de largo y el `COMMIT` confirma el `DELETE` previo. La tablet queda sin instantánea, y el semáforo rechazaría todo como “Producto inexistente”.

Cómo se comprobó. En una base SQLite de prueba, fuera del repositorio, se cargó la instantánea inicial (3 productos) y se recargó con un CSV que repetía el producto 1. Antes de corregir: error `UNIQUE constraint failed` y `snapshot_servidor` con **0** filas.

Corrección. Se agregó `.bail on` al inicio del script y `.bail off` al final. Ante un error, `sqlite3` se detiene antes del `COMMIT` y la transacción se descarta. Después de corregir, con el mismo CSV: el mismo error, pero `snapshot_servidor` conserva sus **3** filas. Prueba de regresión: los casos C2 a C6 dan los mismos resultados que antes. El `.bail off` final evita que el error intencional de C6 corte la ejecución de `casos_2c.sql`.

F09 — Un producto que desaparece del servidor (Unidad 8) · Descartado

Qué dijo la IA. El upsert de `recargar_snapshot.sql` solo actualiza los productos presentes en el CSV. Si un producto desaparece del servidor, quedaría en la tablet como disponible.

Cómo se comprobó. En `foodstore_tp8_feria`, con la Coca 1.5L dada de baja: hay 3 productos y la vista `v_snapshot_movil` exporta 3 filas (1 dada de baja). La vista no filtra por `activo`, así que la baja lógica viaja a la tablet como `disponible = 0`. El intento de borrar físicamente la Coca falló por la clave foránea `fk_detalle_pedido_producto` (`ON DELETE RESTRICT`). No hay productos sin ventas (0 filas).

Decisión. Descartado. En este diseño un producto no desaparece del CSV: se da de baja lógicamente y sigue viajando. **Límite:** solo podría pasar con un producto nuevo, sin ventas, borrado físicamente en vez de darse de baja. Hoy no existe ninguno.